OpenClaw Visual Distance Measurement

OpenClaw Visual Distance Measurement

1. Course Content
   - Course Overview
   - Learning Objectives
2. Preparation
3. Visual Distance Measurement Case Practice
   - Step 1: Issue Distance Measurement Command
   - Step 2: Point Verification Image Troubleshooting
   - Developer Exclusive CLI Command Line Debugging
4. Core Source Code Analysis
   - `GetTargetDist()` — Semantic Anti-projection Ranging Core Logic
5. Common Problems and Optimization Suggestions

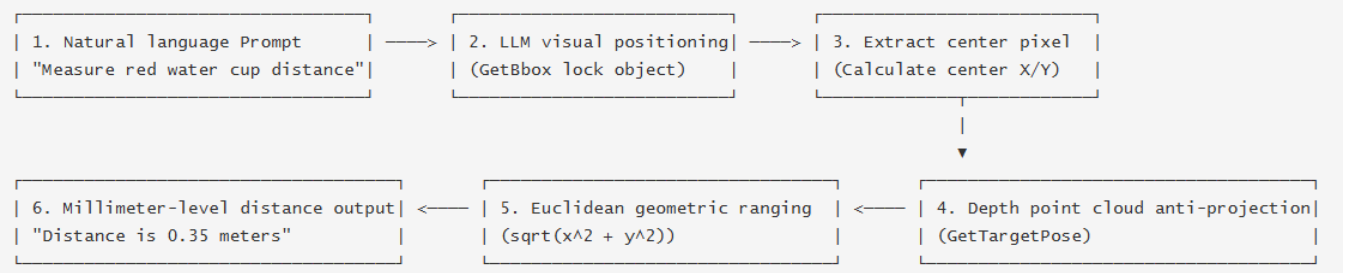
1. Course Content

Course Overview

In robot autonomous operations and environmental interaction, **precise distance perception** is the prerequisite for all advanced actions (such as collision avoidance, autonomous approach, target grasping, safe following).

Traditional ranging relied on ultrasonic sensors or handheld laser rangefinders, which share a common limitation: they must mechanically aim at a single forward direction, and completely cannot understand "which object is being measured". OpenClaw combines multimodal large model with RGB-D depth camera's **semantic visual ranging (Semantic Visual Ranging)**, giving the robot **"smart spatial sense that measures wherever you point"**:

1. **Understands what you want to measure (natural language target locking)**:
   The user doesn't need to manually select with mouse, just naturally express the command (e.g., *"Measure the distance of the red water cup in front"*, *"See how far the yellow cube is from you"*). The large model first locks the 2D pixel range of that object in the view.
2. **Can calculate 3D physical distance (point cloud spatial anti-projection)**:
   The system extracts the target center point pixels, combines the camera factory-calibrated internal focal length and depth point cloud model, uses perspective inverse transformation to instantly inverse solve the target's 3D real-world spatial coordinates (X, Y, Z), and uses the Pythagorean theorem to calculate the horizontal Euclidean straight-line distance.



Learning Objectives

1. Master the method of achieving specified target visual ranging through natural language interaction using `GetTargetDist` tool.
2. Deeply understand the complete mathematical solving process from 2D image pixel coordinates anti-projected to 3D spatial coordinate system.
3. Be able to diagnose visual positioning accuracy through the system's automatically generated verification image (`verify_image.png`) and optimize Prompt description.

---

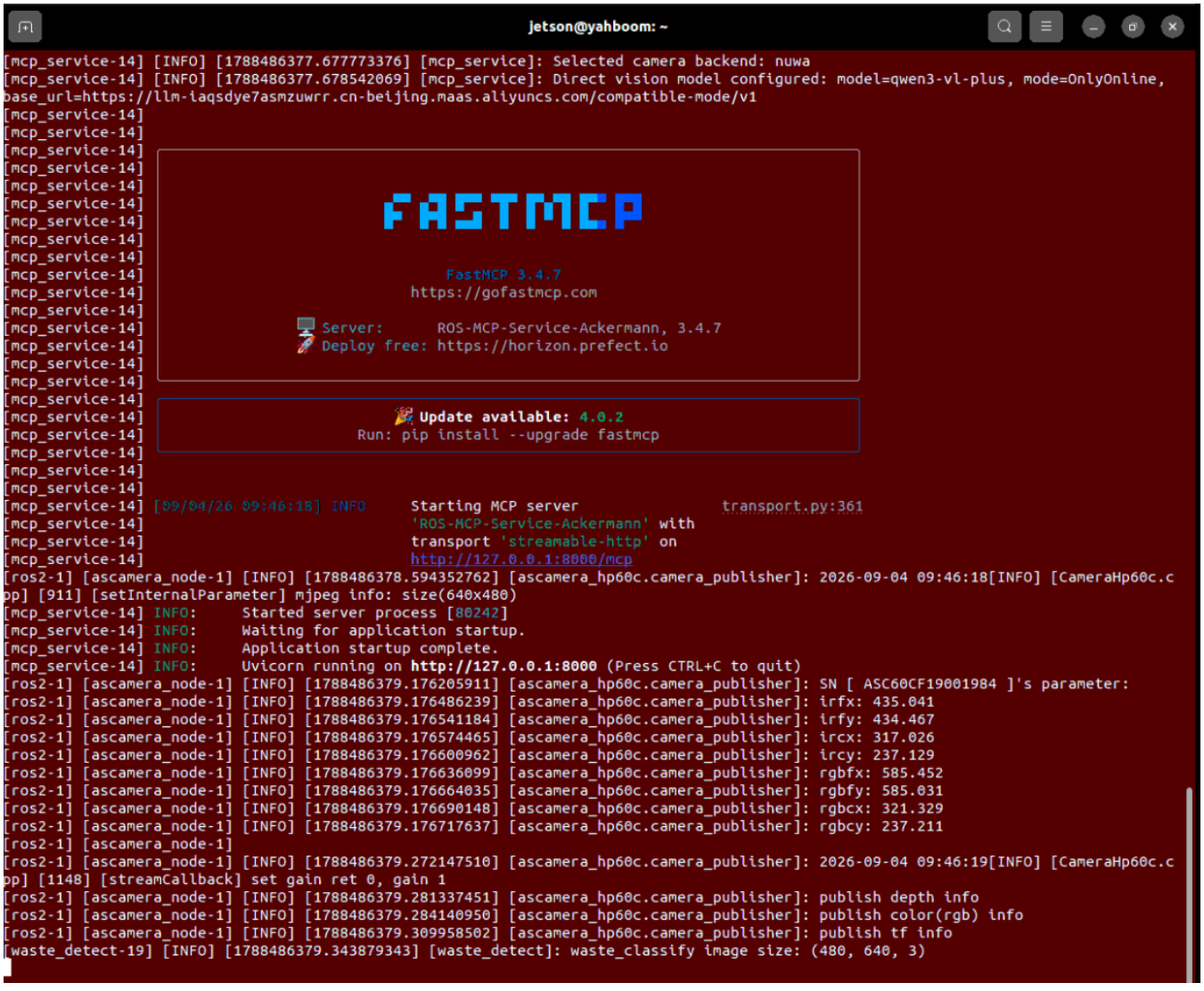
2. Preparation

- Start core nodes:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py
```

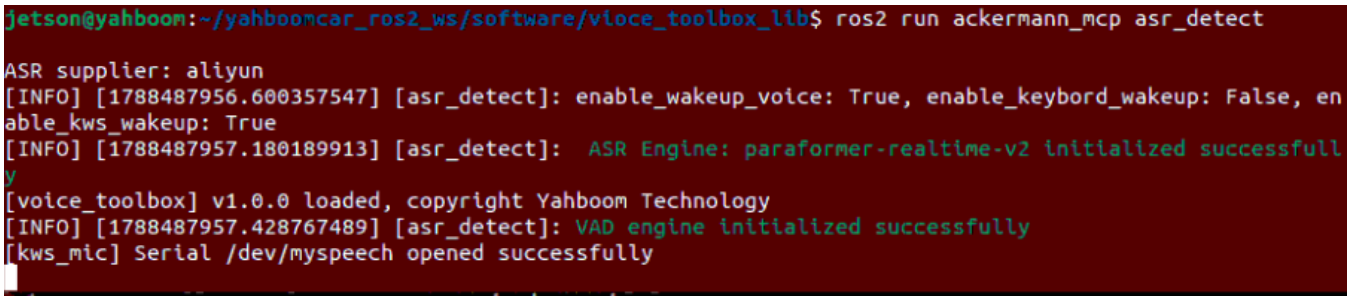
- If you need voice broadcast replies:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py   launch_openclaw_bridge:=true
```



- For voice dialogue interaction:

```
ros2 run ackermann_mcp asr_detect
```



3. Visual Distance Measurement Case Practice

> **Tip**
> Instructions can be expressed naturally according to your actual needs. Here we use WebChat as an example.

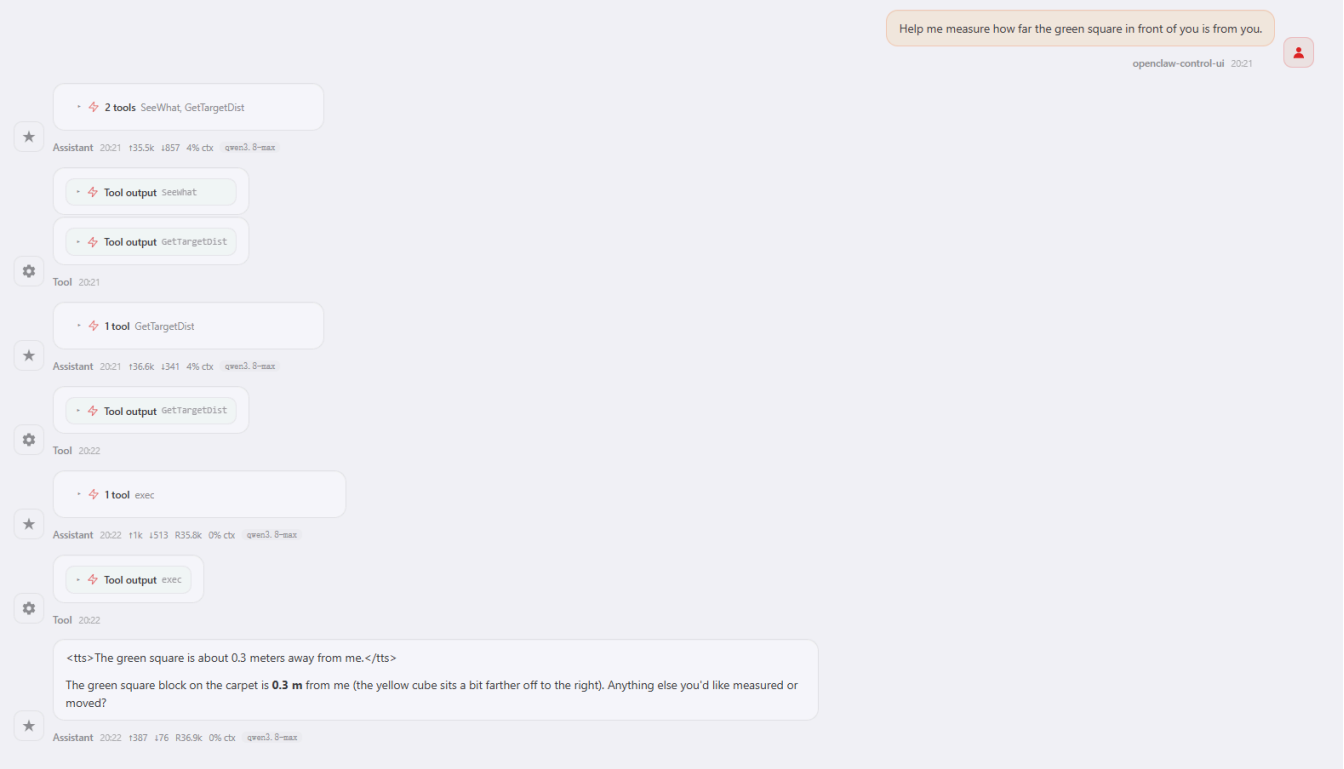
Core calling tool:

Tool Name	Input Parameters	Core Function
`GetTargetDist`	`--query` (target object natural language description)	Circle the object in the view based on description and calculate its straight-line distance relative to the car body reference origin

Step 1: Issue Distance Measurement Command

Enter the distance measurement command in the WebChat interface, for example:

> "**Help me measure how far the yellow block in front of you is from you**"

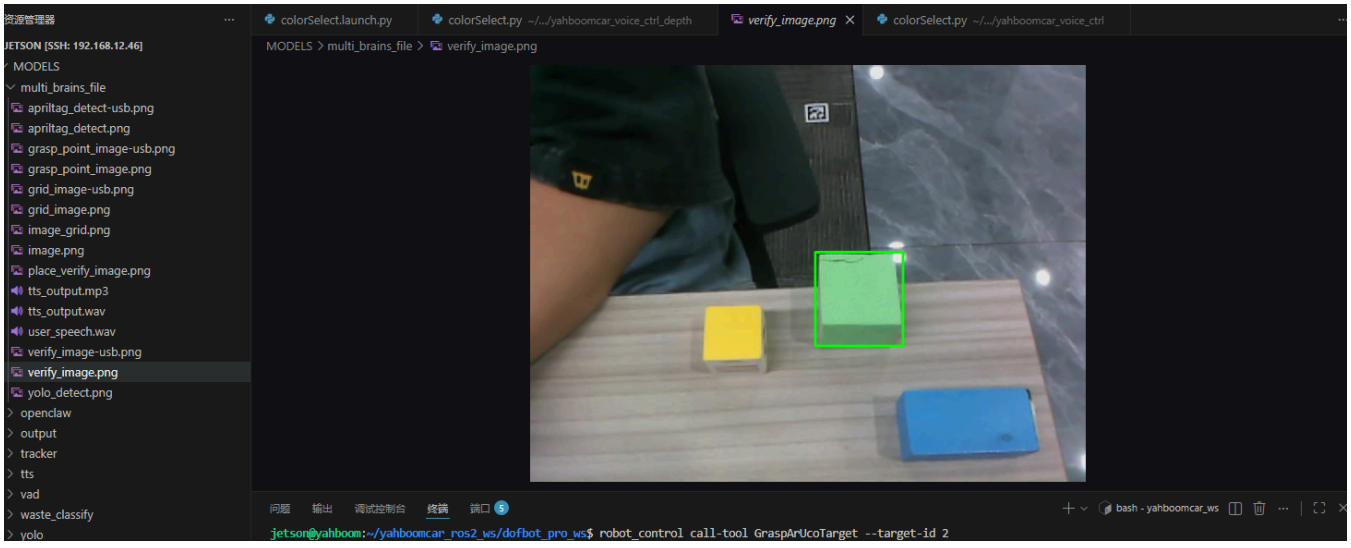


OpenClaw quickly responds, calls the underlying visual distance measurement service, and clearly answers in the session: the target object is approximately `0.35 meters` (35 cm) from the car body.

Step 2: Point Verification Image Troubleshooting

When the `GetTargetDist` tool is called, the system automatically generates a **verification image** in the background that intuitively shows the large model-locked target object in the view:

```
~/MODELS/multi_brains_file/verify_image.png
```

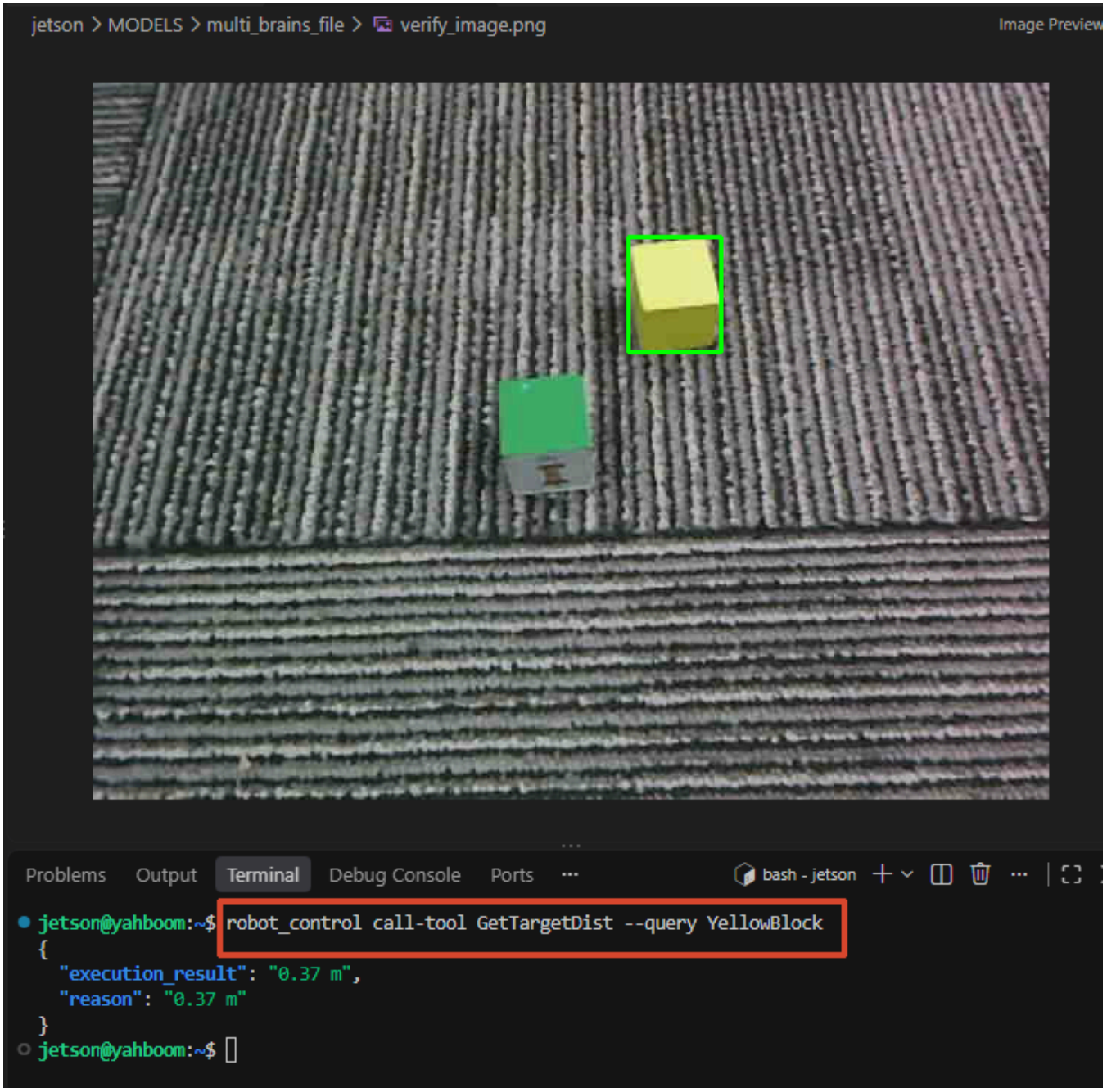


> **Usage tip**: If the measured distance is clearly off from actual, first open this image to check if the large model accurately circled the target. If there are overlapping objects, you can add more precise directional description in the command (e.g., "the yellow cube closest to me").

Developer Exclusive CLI Command Line Debugging

Without opening the web front-end, you can also directly use CLI to debug this tool in the terminal:

```
robot_control call-tool GetTargetDist --query YellowBlock
```



4. Core Source Code Analysis

- Source code file path:

```
~/yahboomcar_ros2_ws/dofbot_pro_ws/src/ackermann_mcp/ackermann_mcp/action_controller.py
```

`GetTargetDist()` — Semantic Anti-projection Ranging Core Logic

```
def GetTargetDist(self, query: str) -> list[bool, float | str]:
    '''Natural language semantic target visual ranging interface'''
    # Step 1: Call large model target detection to get bounding box (Bounding Box) of object
matching query description
    res = self.GetBbox(query)
    if not res[0]:
        return [False, res[1]]

    # Step 2: Parse bounding box coordinates dictionary, calculate object's center pixel
coordinates
    import ast
    bounding_box = ast.literal_eval(res[1])
    target_center_x = (bounding_box['x2'] + bounding_box['x1']) / 2.0
    target_center_y = (bounding_box['y2'] + bounding_box['y1']) / 2.0

    # Step 3: Check if underlying 3D spatial pose anti-projection service is ready
    if not self.get_pose_client.wait_for_service(timeout_sec=8.0):
        msg = "GetTargetPose service not available"
        self.get_logger().error(msg)
        return [False, msg]

    # Step 4: Pass center pixel into service, combine camera depth internal parameters to
inverse solve 3D spatial coordinates (X, Y, Z)
    request = GetTargetPose.Request(
        x=float(target_center_x),
        y=float(target_center_y),
        radius=self.target_circle
    )
    result: GetTargetPose.Response = self.get_pose_client.call(request)
    if not result.success:
        return [False, result.message]

    # Step 5: Use Pythagorean theorem to calculate horizontal Euclidean distance, keep two
decimal places
    import math
    distance = round(
        math.sqrt((result.pose.position.x ** 2) + (result.pose.position.y ** 2)),
        2
    )
    return [True, distance]
```

---

5. Common Problems and Optimization Suggestions

1. **Problem: Ranging result returns error or prompts "target does not exist"?**
    - **Optimize**: Check if ambient light is too dark, or if the target is outside the camera's effective field of view. Try to use vocabulary with obvious features to describe objects (e.g., "the green mineral water bottle closest to me" is easier for the large model to accurately lock than simply saying "bottle").

2. **Problem: The measured distance has a few centimeters error compared to tape measure actual measurement?**
    - **Explanation**: RGB-D depth camera's optimal ranging interval is between `0.2 meters ~ 2.0 meters`. If the object is too close (< 0.15m), it enters the camera blind zone and cannot image; if the object surface is highly reflective material (such as smooth glass or mirror stainless steel), it is recommended to attach a layer of sandpaper to obtain the most precise point cloud ranging result.